Trabalho: Algoritmo de Dijkstra

Flavio Jorge

Outubro de 2026

1 Introdução

A proposta deste trabalho é escolher um objeto de estudo a ser utilizado como *dataset* para a construção de um grafo valorado.

Em seguida, o algoritmo de Dijkstra será implementado e aplicado ao grafo resultante do objeto de estudo.

2 Escolher o problema

Quando comecei a aprender grafos, minha primeira intuição, ao ver a "cara" de um grafo com vários vértices e arestas, foi a árvore de digievoluções, estrutura presente nos jogos *Digimon*.

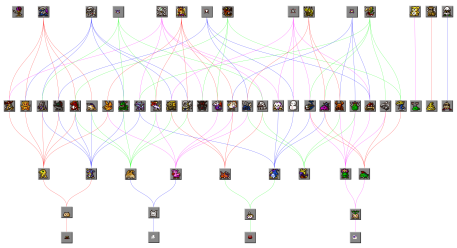


Figure 1: Árvore de digievoluções do Digimon World 1.

Nesses jogos, os monstros digitais, chamados de Digimons, possuem a capacidade de evoluir, ou seja, de mudar de forma para outros Digimons. Sendo assim, uma digievolução nada mais é do que uma aresta direcionada entre um par de Digimons, predecessor e sucessor, à qual está associado, dentro da mecânica do jogo, um custo referente a essa mudança.

3 Conseguir dados

Para obter a lista de Digimons, suas respectivas digievoluções e custos, utilizei um site conhecido dentro da comunidade de fãs do jogo [1], que calcula o custo

de um Digimon de origem até um Digimon de destino.

Para obter o dataset, precisei extrair a lógica do código-fonte do site [3]. Para isso, construí dois scripts no repositório [2], chamados *parse_digimon.py* e *parse_costs.py*, que geram os datasets *digimon.json*, contendo a lista de digievoluções, e *costs.json*, contendo a lista de custos.

3.1 Custo das arestas

Dentro do jogo, diferentemente de outros jogos em que existe um nível que é incrementado ao longo da partida, não existe tal métrica em Digimon. Sendo assim, um Digimon evolui quando cumpre um conjunto de requisitos, como uma certa quantidade de pontos de ataque e defesa, além de estar dentro de uma faixa de peso ou ser muito disciplinado.

A fim de simplificar o cálculo posterior do algoritmo, mesmo que não seja a forma perfeita e absoluta de calcular o custo, defini a quantidade de requisitos como **pontos-chave**. Essa métrica é adotada em alguns jogos mais simples de Digimon, que condensam a soma dos requisitos em um único valor, de modo que, quanto mais requisitos, maior o valor do vértice.

4 Construir o grafo em memória

Com o dataset pronto, aproveitei a estrutura base de grafo, implementada em Python, que utilizei em outros trabalhos desta disciplina, e adicionei o valor do peso do vértice na modelagem.

Para construir o grafo, utilizei o seguinte método, pertencente à classe disponível no repositório [2]:

```
    def create_graph(self, evolution_data, cost_data):
        evolutions = json.loads(evolution_data)
        costs = json.loads(cost_data)
        node_map = {}

        for digimon, targets in evolutions.items():
            if digimon not in node_map:
                node_digimon = self.add_node(digimon)
                node_map[digimon] = node_digimon
            else:
                node_digimon = node_map[digimon]

            for target in targets:
                if target not in node_map:
                    node_target = self.add_node(target)
                    node_map[target] = node_target
                else:
                    node_target = node_map[target]

                cost = costs.get(target, 0)
```

```
21                  self.add_edge(node_digimon, node_target,
                        cost)
```

Listing 1: Algoritmo para construir o grafo

Esse grafo, no formato de lista de adjacências, fica da seguinte forma:

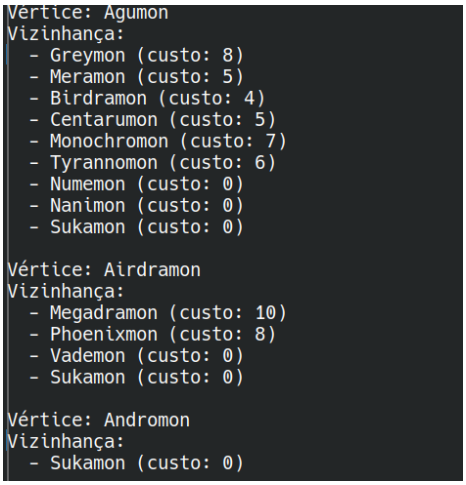


Figure 2: Exemplo de impressão do grafo.

5 Executar o Dijkstra

Com a lista de adjacências pronta, bastou implementar o algoritmo de Dijkstra utilizando *heap* e percorrendo as digievoluções a partir de uma origem.

```
1       def dijkstra(self, origin_value):
2           for u in self.nodes:
3               u.d = float("inf")
4               u.pi = None
5               u.color = "white"
6
7           origin_node = next((n for n in self.nodes if n.value
                == origin_value), None)
8           if origin_node is None:
9               print(f"Origin {origin_value} not found in graph
                    .")
10              return
11
12          origin_node.d = 0
13
14          counter = 0
15          pq = [(0, counter, origin_node)]
```

```
16
17          while pq:
18              dist_u, _, u = heapq.heappop(pq)
19
20              if u.color == "black":
21                  continue
22
23              u.color = "black"
24
25              for v, cost in u.neighbors:
26                  if v.color != "black":
27                      new_dist = u.d + cost
28                      if new_dist < v.d:
29                          v.d = new_dist
30                          v.pi = u
31                          v.color = "gray"
32                          counter += 1
33                          heapq.heappush(pq, (new_dist,
                                counter, v))
```

Listing 2: Algoritmo de Dijkstra com *heap*

6 Conclusão

Esse trabalho tem um valor pessoal muito grande para mim, por envolver um jogo com o qual tenho uma relação de longa data. Além disso, foi uma oportunidade de pegar conhecimentos que adquiri empiricamente ao longo do tempo sobre as digievoluções e complementá-los com uma abordagem mais técnica. Ao representar essa árvore como um grafo valorado, pude aplicar o algoritmo de Dijkstra a um contexto que já conhecia, entendendo melhor como ele funciona na prática e explorando o problema por uma perspectiva diferente.

References

[1] *Digimon World Evolution*. URL: `https://phoenix-staffel.de/digimon/DigimonWorld/evolution.html` (visited on 10/18/2026).

[2] Flavio Diniz. *Digimon Graph Study*. URL: `https://github.com/flavin27/digimon-graph-study` (visited on 10/18/2026).

[3] SydMontague. *DigimonWorldWebsite*. GitHub. URL: `https://github.com/SydMontague/DigimonWorldWebsite/tree/master` (visited on 10/18/2026).